

HelixCluster

HelixCluster

AI कंप्यूटिंग के लिए एक वितरित ऑपरेटिंग सिस्टम — डेटासेंटर जीपीयू से लेकर एज हैडहेल्ड डिवाइस तक, एक ही नियंत्रण प्लेन के तहत।

सारांश

Helix क्लस्टर ओएस एक अगली पीढ़ी का वितरित ऑपरेटिंग सिस्टम है जो विषम नोड्स — डेटासेंटर जीपीयू से लेकर एज एसबीसी और हैडहेल्ड डिवाइस तक — पर कंप्यूटिंग का समन्वय करता है। यह एचपीसी शेड्यूलिंग, कंटेनर ऑर्किस्ट्रेशन, AI/ML इंफरेंस, फेडरेटेड मल्टी-क्लस्टर ऑपरेशन और सुरक्षित मल्टी-टेनेंट सत्रों को एक ही नियंत्रण प्लेन के तहत एकीकृत करता है।

संक्षिप्त विवरण

एक Go-आधारित वितरित ओएस / GPU-साझाकरण कंप्यूट क्लस्टर। यह एचपीसी शेड्यूलिंग (ओमेगा-मॉडल द्विस्तरीय शेड्यूलर), कंटेनर ऑर्किस्ट्रेशन, AI इंफरेंस रूटिंग, फेडरेशन और विषम नोड्स पर सुरक्षित मल्टी-टेनेंट सत्रों को एकीकृत करता है, जिसे SWIM गॉसिप और राफ्ट सहमति द्वारा समन्वित किया जाता है, साथ ही पोस्ट-क्वांटम एंड-टू-एंड एन्क्रिप्शन के साथ।

विस्तृत विवरण

Helix क्लस्टर ओएस कंप्यूटिंग वर्कलोड्स को अत्यधिक विषम हार्डवेयर — डेटासेंटर जीपीयू, एज सिंगल-बोर्ड कंप्यूटर, यहाँ तक कि हैडहेल्ड डिवाइस — पर एक ही नियंत्रण प्लेन के तहत समन्वित करता है। यह A100 के एक रैक और एसबीसी के एक समूह को एक ही संबोधित करने योग्य फैब्रिक के रूप में देखता है, न कि दर्जनों असंगत द्वीपों के रूप में। यह एक Go वर्कस्पेस (एक मोनोरेपो और गिट सबमॉड्यूल्स के साथ) है जो सात परतों वाले स्टैक को लागू करता है — L0 हार्डवेयर सबस्ट्रेट से लेकर L7 फेडरेशन और ऑब्जर्वेबिलिटी तक — जिसे चौदह नियंत्रण-प्लेन माइक्रोसर्विसेज द्वारा समन्वित किया जाता है। नोड सदस्यता को SWIM गॉसिप और डिस्कवरी के माध्यम से ट्रैक किया जाता है ताकि फैब्रिक नोड्स के जुड़ने और छोड़ने पर स्वतः ठीक हो सके; दृढ़ता से संगत स्थिति राफ्ट सहमति पर आधारित होती है, जिसे प्रति-शार्ड राफ्ट समूहों के रूप में व्यवस्थित किया जाता है, जिसमें गति के लिए लीज़होल्डर-लोकल रीड्स और साझा स्थिति को दूषित होने से बचाने के लिए STONITH फेंसिंग शामिल है।

वर्कलोड प्लेसमेंट एक ओमेगा-मॉडल द्विस्तरीय शेड्यूलर के माध्यम से संचालित होता है — आशावादी समवर्तीता, क्लासएड मिलान, गैंग शेड्यूलिंग, मूल्य-गुणक प्रीएम्प्शन और बाधा-आधारित प्लेसमेंट — और फिर एक क्लासिक एचपीसी शेड्यूलर से भी आगे जाता है: कार्बन-जागरूक और लागत/टीसीओ-जागरूक रूटिंग, क्लाउड पर बर्स्ट ऑटोस्केलिंग, और मार्केटप्लेस एडेप्टर (अकाश, io.net, रनपॉड, AWS स्पॉट, च्यूट्स) जो स्थानीय संसाधनों के समाप्त होने पर किराए की क्षमता पर कार्य को स्थानांतरित करने की अनुमति देते हैं।

अंतिम उपयोगकर्ता इस मशीनरी को सीधे नहीं देखते; वे एक स्वच्छ सत्र मॉडल (कंप्यूट आवंटन), एक इंटरैक्टिव WebSocket/PTY टर्मिनल, एक आंतरिक AI-इंफ्रेंस रूट और पूल-उपयोग रीड्स के माध्यम से बातचीत करते हैं। सुरक्षा एक प्राथमिक परत है, न कि बाद में जोड़ी गई चीज़: SPIFFE पहचान, डिवाइस प्रमाणीकरण (चुनौती/प्रतिक्रिया, GPU-कार्य का प्रमाण, सीलिंग), निर्यात-नियंत्रण केवाईसी गेट, और एक पोस्ट-क्वांटम एंड-टू-एंड एन्क्रिप्टेड ट्रांसपोर्ट जो X25519 + ML-KEM-768 हाइब्रिड कुंजी विनिमय पर आधारित है, जिसमें AEAD रिकॉर्ड सुरक्षा और रीप्ले अस्वीकृति शामिल है — इसे इस तरह से डिज़ाइन किया गया है कि आज का कैप्चर किया गया ट्रैफ़िक कल के क्वांटम विरोधी के खिलाफ भी गोपनीय बना रहे। शुद्धता को केवल दावा नहीं किया जाता, बल्कि प्रदर्शित किया जाता है: नियतात्मक सिमुलेशन परीक्षण (फाउंडेशनडीबी-शैली के सीडेड रन, दोष इंजेक्शन, नेटवर्क सिमुलेशन, बाइट-फॉर-बाइट रीप्ले और पोर्क्यूपाइन लीनियराइज़ेबिलिटी चेकर) वितरित विफलताओं को मांग पर पुनः उत्पन्न करता है, और अनिवार्य युग्मित म्यूटेशन परीक्षण साबित करता है कि गार्ड टेस्ट वास्तव में प्रभावी हैं। वास्तुकला और दस्तावेज़ों की सत्यता को यांत्रिक लिंट्स द्वारा सुनिश्चित किया जाता है जो बिल्ड को तुरंत विफल कर देते हैं जब वास्तविकता और दस्तावेज़ीकरण में अंतर आता है।

सामग्री

हमने इसे क्यों बनाया

AI और HPC वर्कलोड को अत्यधिक भिन्न हार्डवेयर स्तरों पर चलाने के लिए—अलग-अलग शेड्यूलर, ऑर्किस्ट्रेटर और इंफ्रेंस स्टैक को जोड़ने की ज़रूरत के बिना—और यह सुनिश्चित करने के लिए कि हर रिलीज़ की गई सुविधा वास्तविक अंतिम-उपयोगकर्ता व्यवहार को साबित करे (कभी भी स्टब्स पर ग्रीन टेस्ट नहीं), और हर OS-विशिष्ट क्षमता प्रत्येक प्लेटफ़ॉर्म पर वास्तविक नेटिव सुविधा का उपयोग करे (कोई Linux-ओनली मॉक नहीं)। रेपो के गवर्नेंस में उद्धृत प्रेरक समस्या है—“टेस्ट पास हो जाते हैं, लेकिन सुविधा वास्तव में काम नहीं करती”—यह विफलता मोड, जिसे इस प्रोजेक्ट को स्पष्ट रूप से समाप्त करने के लिए बनाया गया है।

यह क्यों गेम-चेंजर है

यह पाँच चीज़ों को, जो सामान्यतः पाँच अलग-अलग स्टैक होती हैं—HPC शेड्यूलिंग, कंटेनर ऑर्किस्ट्रेशन, AI इंफ्रेंस, मल्टी-क्लस्टर फ़ेडरेशन और सुरक्षित मल्टी-टेनेंट सत्र—एक ही कंट्रोल प्लेन में समाहित कर देता है, जो डेटासेंटर GPU से लेकर एज हैंडहेल्ड डिवाइस तक फैला होता है। और यह ऐसा कठोरता बजट के साथ करता है, जो आमतौर पर विशेष इंफ्रास्ट्रक्चर के लिए आरक्षित होता है: फ़ॉर्मल-मेथड्स-ग्रेड शुद्धता (TLA+ स्पेक्स, नियतात्मक सिमुलेशन, लीनियराइज़ेबिलिटी जाँच) और पोस्ट-क्वांटम गोपनीय ट्रांसपोर्ट—वे गारंटी जो अधिकांश ऑर्किस्ट्रेटर प्रयास भी नहीं करते। तकनीकी भिन्नता के अलावा, लागत- और कार्बन-जागरूक प्लेसमेंट के साथ-साथ क्लाउड-मार्केटप्लेस बर्स्ट इसे आर्थिक

लीवर भी बनाते हैं—शेड्यूलर सस्ती, हरित या अतिरिक्त क्षमता का स्वचालित रूप से पीछा कर सकता है, जिससे वही वर्कलोड कम लागत पर और कम उत्सर्जन के साथ चलता है, बिना किसी को जॉब को फिर से लिखने की ज़रूरत पड़े।

क्या है नवीन

- **नियतात्मक सिमुलेशन परीक्षण (DST)** — एक सीडेड, पूरी तरह पुनरुत्पादित सिमुलेटर जो दोष, क्लॉक स्क्यू और नेटवर्क विभाजन इंजेक्ट करता है, उन्हें बाइट-दर-बाइट रीप्ले करता है, और परिणाम को पोर्क्यूपाइन लीनियराइज़ेबिलिटी चेकर से गुज़ारता है, ताकि एक बार पकड़ा गया हाइज़नबग हमेशा मांग पर पुनरुत्पादित किया जा सके।
- **ओमेगा-मॉडल द्विस्तरीय शेड्यूलर** — आशावादी-scurrency प्लेसमेंट क्लासएड मैचिंग, गैंग शेड्यूलिंग और वैल्यू-मल्टीप्लायर प्रीएम्प्शन के साथ, वह साझा-स्थिति डिज़ाइन जो कई शेड्यूलरों को एक क्लस्टर के विरुद्ध कमिट करने की अनुमति देता है बिना किसी केंद्रीय बाधा के।
- **पोस्ट-क्वांटम E2EE / गोपनीय इंफ़ॉरस वायरिंग** — X25519 + ML-KEM-768 हाइब्रिड कुंजी विनिमय, प्रति-अनुरोध रिस्पॉन्स-कीपेयर बाइंडिंग और AEAD के साथ रीप्ले अस्वीकृति (क्रिप्टो प्रिमिटिव वास्तविक और परीक्षित हैं; पूर्ण गोपनीय मल्टी-नोड राउंड-ट्रिप स्पष्ट रूप से योजनाबद्ध/गेटेड है)।
- **प्रमाण-आधारित विश्वास** — नोड्स को साबित करना होता है कि वे क्या हैं: SPIFFE पहचान, प्रूफ़-ऑफ़-GPU-वर्क, डिवाइस सीलिंग, एक्सपोर्ट-कंट्रोल KYC गेट, और EU AI अधिनियम अनुपालन दस्तावेज़-जनरेशन, ताकि विश्वास नेटवर्क स्थिति से मान लिया जाए बल्कि साक्ष्य द्वारा अर्जित किया जाए।
- **लागत- और कार्बन-जागरूक ऑर्किस्ट्रेशन** — TCO मॉडलिंग, कार्बन-जागरूक प्लेसमेंट, क्लाउड-बस्ट, N+K फ़ेलओवर रिज़र्व, और क्लाउड-मार्केटप्लेस एडेप्टर, जो मूल्य और उत्सर्जन को शेड्यूलिंग के प्राथमिक इनपुट बनाते हैं, न कि बाद की सोच।
- **मल्टी-राफ़्ट सहमति** — प्रति-शार्ड राफ़्ट समूह लीज़होल्डर-लोकल रीड्स के साथ कम-विलंबता स्थिरता के लिए, STONITH फ़ेंसिंग (IPMI / EC2 / Azure / SBD) द्वारा समर्थित, ताकि अटका हुआ नोड निर्णायक रूप से हटाया जाए, न कि स्थिति को भ्रष्ट करने के लिए छोड़ दिया जाए।
- **एंटी-ड्रिफ़्ट मैकेनिकल लिंट्स** — archLint बिल्ड को तुरंत विफल कर देता है जब कोई दस्तावेज़ीकृत घटक किसी ऐसे पैकेज पथ पर मैप होता है जो मौजूद नहीं है, और एक डॉक्स-चेन इंजन मार्कडाउन / HTML / PDF / DOCX को बाइट-संगत रखता है, ताकि दस्तावेज़ीकरण चुपचाप कोड के बारे में झूठ न बोल सके।

सामग्री

सबसे बड़ी तकनीकी चुनौतियाँ और उनका समाधान

- **“PASS-bluff” (गैर-कार्यात्मक सुविधाओं पर परीक्षण पास होना)**। वह विफलता मोड जिसे पूरा प्रोजेक्ट खत्म करने के लिए बनाया गया है: स्टब्स पर हरी झंडी। अनिवार्य युग्मित म्यूटेशन परीक्षण से हल किया गया — हर कार्य आइटम के साथ एक नामित गार्ड परीक्षण जुड़ा होता है, जो स्वतंत्र कोड म्यूटेशन के तहत विफल होना चाहिए, तभी आइटम को पूरा माना जाता है। इससे यह सिद्ध होता है कि पास होने वाला परीक्षण वास्तविक व्यवहार की जाँच करता है, न कि किसी मॉक की।
- **क्रॉस-प्लेटफ़ॉर्म समानता (केवल Linux के लिए मॉक नहीं)**। एक साझा इंटरफ़ेस से हल किया गया, जिसे बिल्ड टैग्स के माध्यम से वास्तविक प्रति-ओएस सुविधाओं में

विभाजित किया गया — Linux cgroup / /proc / कर्नेल WireGuard, macOS sysctl / vm_stat / IOKit / wireguard-go — और फिर एक स्वतंत्र ओएस ऑरेकल के विरुद्ध क्रॉस-चेक किया गया, ताकि हर प्लेटफॉर्म वास्तविक नेटिव स्थिति रिपोर्ट करे, न कि Linux की काल्पनिक धारणा।

- **विफलताओं के तहत वितरित शुद्धता**। नियतात्मक सिमुलेशन परीक्षण और एक रैखिकता जाँचकर्ता से हल किया गया, जो विभाजन, क्लैश और क्लॉक स्क्यू को उत्पन्न और पुनः चलाता है, साथ ही TLA+ औपचारिक विनिर्देशों द्वारा समर्थित, जो सर्वसम्मति और शेड्यूलिंग इन्वेरिएंट्स को कोड की एक भी पंक्ति लिखे जाने से पहले निर्धारित करते हैं।
- **दस्तावेजीकरण और आर्किटेक्चर में विचलन**। archlint से हल किया गया, जो किसी भी दस्तावेजीकृत-परंतु-अस्तित्वहीन मैप्ड पैकेज पर बिल्ड को विफल कर देता है, और एक docs_chain वेरिफाई गेट के साथ, जिसमें कोई बचाव मार्ग नहीं — विचलन बिल्ड ब्रेक होता है, पुराना विकी पृष्ठ नहीं।
- **अपूर्ण कार्य का ईमानदार दायरा**। गोपनीय मल्टी-नोड अनुमान राउंड-ट्रिप को जानबूझकर एक टिकट के पीछे गेट किया गया है और “अभी तक एंड-टू-एंड सत्यापित नहीं” के रूप में लेबल किया गया है, बजाय इसके कि इसे शिफ्ट के रूप में प्रस्तुत किया जाए — वही अनुशासन जो अभी तक नहीं किया गया है, उस पर भी लागू होता है जो किया जा चुका है।

तकनीकी स्टैक

- **Go (go.mod: 1.25 / टूलचेन 1.26.4)** — लगभग 30-मॉड्यूल वाले कार्यक्षेत्र में नियंत्रण-तल भाषा; चुना गया सस्ते गोरूटीन कंकरेसी और स्टैटिक बायनेरिज के लिए, जो डेटासेंटर से एज तक समान रूप से तैनात होते हैं।
- **Zig (0.14+) + C/C++** — वहाँ उपयोग किया गया जहाँ Go का रनटाइम बाधा बनता है: निम्न-स्तरीय सिस्टम प्रिमिटिव्स और GPU कर्नेल्स के लिए, जिन्हें हार्डवेयर पर नियतात्मक, आवंटन-मुक्त नियंत्रण की आवश्यकता होती है।
- **gRPC + Protocol Buffers** — हर इंटर-सबसिस्टम API (api/v1/) एक टाइप्ड, वर्जनयुक्त अनुबंध है, ताकि चौदह माइक्रोसर्विसेज बिना एक-दूसरे को तोड़े या वायर फॉर्मेट्स को मैनुअली तैयार किए विकसित हो सकें।
- **Raft (etcd-raft) + SWIM गॉसिप** — जानबूझकर विभाजन: Raft उस स्थिति को ले जाता है जिसे मजबूत स्थिरता की आवश्यकता होती है, जबकि SWIM गॉसिप सदस्यता और खोज को बड़े पैमाने पर संभालता है, जहाँ सर्वसम्मति बहुत भारी होती।
- **PostgreSQL 16, Redis 7 क्लस्टर, etcd v3.5, SQLite** — हर काम के लिए सही स्टोर: टिकाऊ रिलेशनल स्थिति के लिए PostgreSQL, हॉट कैश के लिए Redis, समन्वय के लिए etcd, और नोड-लोकल HXC कार्य-आइटम रजिस्ट्री के लिए एम्बेडेड SQLite।
- **NATS 2.10 (JetStream), Kafka 4.0 (KRaft), RabbitMQ 3.13** — तीन मैसेजिंग बैकबोन तीन प्रकार के ट्रैफिक के लिए: तेज़ आंतरिक इवेंटिंग के लिए NATS/JetStream, टिकाऊ उच्च-थ्रूपुट स्ट्रीम्स के लिए Kafka, और क्लासिक ब्रोकर सिमेंटिक्स के लिए RabbitMQ।
- **WireGuard मेश + ML-KEM-768/X25519 + AES-256-GCM/ChaCha20-Poly1305 + HKDF** — WireGuard एक हल्के नोड-टू-नोड मेश के लिए, जिसे हाइब्रिड पोस्ट-क्वांटम हैंडशेक और AEAD रिकॉर्ड्स में लपेटा गया है, ताकि ट्रांसपोर्ट क्लासिकल और क्वांटम दोनों हमलावरों के खिलाफ गोपनीय रहे।
- **SPIFFE + JWT (HS256) + स्कोप-आधारित RBAC + OPA** — स्तरित पहचान और प्राधिकरण: SPIFFE वर्कलोड पहचान के लिए, JWT टोकन के लिए, स्कोप-आधारित RBAC मोटे एक्सेस के लिए, और OPA कोड के रूप में बारीक नीति व्यक्त करने के लिए।

- **Prometheus v2.50, Grafana 10.4, Jaeger 1.55, W3C ट्रेसिंग** — मेट्रिक्स, डैशबोर्ड और वितरित ट्रेस W3C कॉन्टेक्स्ट प्रोपेगेशन के साथ, ताकि किसी अनुरोध को सेवाओं और हार्डवेयर स्तरों में ट्रैक किया जा सके।
- **HashiCorp Vault 1.16** — रहस्य और कुंजी सामग्री कोड और कॉन्फ़िगरेशन से बाहर रखी गई और ऑडिट के तहत जारी की गई।
- **Docker Compose, Kubernetes (kustomize, हार्डनड securityContext), Helm** — स्थानीय सेटअप के लिए Compose और वास्तविक तैनाती के लिए Kubernetes/Helm हार्डनड सुरक्षा संदर्भों के साथ, एक ही परिभाषा को सभी वातावरणों में प्रमोट किया गया।
- **React + TypeScript + Vite (Node 20+)** — सत्रों, टर्मिनलों और पूल उपयोग के लिए एक तेज़, टाइप-सुरक्षित वेब UI।
- **TLA+** — सर्वसम्मति और शेड्यूलिंग इन्वेरिएंट्स का औपचारिक विनिर्देश, ताकि सबसे कठिन-से-परीक्षण गुणों को कार्यान्वयन से पहले डिज़ाइन स्तर पर सिद्ध किया जा सके।

सामग्री

स्थिति एवं ईमानदारी संबंधी टिप्पणियाँ

- **स्थिति:** विकासाधीन। यह संस्करण प्रारंभिक (0.1.0-dev) है। कई उन्नत सुविधाएँ — पूर्ण गोपनीय बहु-नोड अनुमान चक्र, बाज़ार निपटान और सत्यापन-आधारित शेड्यूलिंग जनसंख्या — भंडार में स्पष्ट रूप से योजनाबद्ध / इंफ्रा-नियंत्रित के रूप में चिह्नित हैं और इन्हें पूरी तरह कार्यशील के रूप में प्रस्तुत नहीं किया गया है। कवरेज आँकड़े स्व-रिपोर्टेड हैं।
- **लाइसेंस:** अधोषित। स्पष्ट रूप से घोषित नहीं; Helm चार्ट के HelixCluster/HelixCluster और helixcluster.io यूआरएल अप्रमाणित प्लेसहोल्डर हैं, जो वास्तविक रिमोट से मेल नहीं खाते।
- **संलग्न LLM स्टैक प्रोजेक्ट्स** (LLMOrchestrator, LLMProvider, LLMsVerifier) डीकपल्ड सबमॉड्यूल हैं, क्लस्टर के भीतर होस्ट किए गए मॉडल सर्वर नहीं।

प्राथमिकता स्तर: Helix-प्राथमिक (LLM-इंफ्रास्ट्रक्चर क्लस्टर — वह कंप्यूटिंग आधार जो अनुमान और कंप्यूट वर्कलोड को होस्ट कर सकता है)। यह HelixTrack के बाद आता है।